Relatório 7: Sensores e atuadores.
Internet das Coisas - Integral
Prof. Gabriel de Freitas Fernandes
2026.2

EQUIPE:
Arthur Peixoto - 170.214
Gabriel Oliva Gurgel - 168.860
Guilherme Samuel dos Santos - 168.867

1 Introdução

Um dos usos mais comuns relacionados aos sistemas embarcados é a aquisição de grandezas físicas (no mundo real) por meio de sensores e o uso dessas informações para o acionamento de atuadores. Nesse contexto, o microcontrolador atua como elemento central, responsável por converter sinais do ambiente em dados digitais e, a partir deles, gerar sinais de controle adequados para o dispositivo que se deseja acionar.

Nesta prática, será realizada uma atividade envolvendo um ESP32, um sensor de luminosidade do tipo *Light Dependent Resistor* (**LDR**) e um servomotor **MG996R**. Em um primeiro modelo, o ESP32 realizará a leitura do LDR por meio de seu conversor analógico-digital (ADC) interno e converterá o valor adquirido em um sinal de controle da posição angular do servomotor. Dessa forma, a posição do servo passa a variar em função da intensidade luminosa incidente sobre o sensor.

Em um segundo momento, o modelo inicial será aprimorado com a inclusão do módulo conversor analógico-digital externo **ADS1115** (com melhor resolução) que se comunica com o microcontrolador por meio do protocolo I2C. A substituição do ADC interno do ESP32 por esse módulo visa melhorar a resolução, a linearidade e a estabilidade das medições, uma vez que o conversor nativo do ESP32 apresenta não linearidades e maior sensibilidade a ruído. Essa etapa permite, ainda, a comparação direta entre as duas abordagens de aquisição e a análise de seu impacto na qualidade do controle do motor.

2 Prática

2.1 ADC interno

O ESP32 dispõe de dois conversores analógico-digitais de aproximação sucessiva (SAR) de 12 bits, denominados **ADC1** e **ADC2**, segundo a sua documentação oficial (**??**). O ADC1 possui oito canais, distribuídos nos pinos GPIO32 a GPIO39, enquanto o ADC2 possui dez canais, mas compartilha recursos com o módulo Wi-Fi e, por isso, tem seu uso restringido quando este está ativo. Para a realização da prática, foi adotado o **ADC1**. Não foi necessário testar outra maneira de ligação.

2.2 ADC externo e comunicação I2C

Embora o ESP32 possua conversores analógico-digitais internos, sua curva de resposta apresenta não linearidades e variações ao longo da faixa de medição. Além disso, o ruído gerado pelos módulos de rádio (por exemplo, Wi-Fi e *Bluetooth Low Energy*) pode se acoplar às leituras, de modo que o uso direto do ADC interno não é recomendável quando se deseja maior precisão. Uma alternativa usada pela comunidade em fóruns é empregar um conversor externo, como o **ADS1115**, conectado ao microcontrolador por meio do protocolo I2C. O ESP32 dispõe de dois controladores I2C, denominados **I2C0** e **I2C1**, segundo a sua documentação pela EspressIf. Para a realização da prática, foi adotada a interface **I2C0**, utilizada por padrão pela biblioteca `Wire`, com o pino **GPIO21** como linha de dados (SDA) e o pino **GPIO22** como linha de relógio (SCL). Essa foi a configuração inicialmente definida pelo grupo, e o módulo **ADS1115** foi endereçado em `0x48`, endereço padrão obtido com o pino ADDR ligado ao GND (**??**). Assim como na primeira parte da prática, não foi necessário testar outra maneira de ligação.

2.3 Montagem Experimental

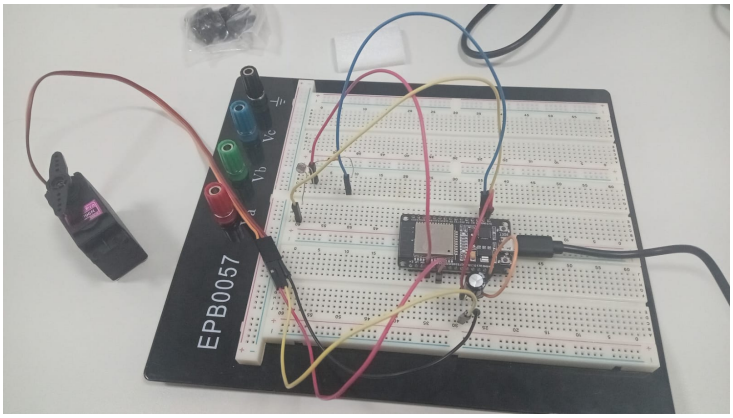


Figura 1: Usando ADC interno

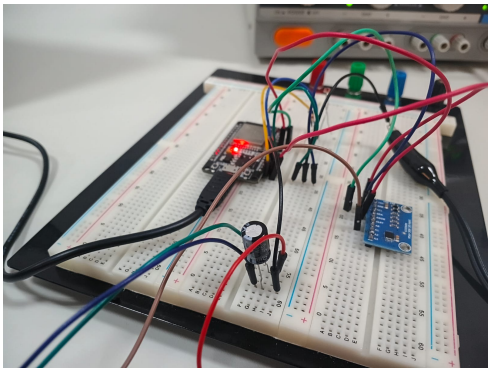


Figura 2: Usando ADC externo

2.4 Código

Para a leitura do sensor, um dos pinos foi configurado como entrada por meio da função `pinMode`, por pertencer ao **ADC1** e ser um pino exclusivo de entrada. A função `analogRead` retorna um valor inteiro de 12 bits, ou seja, na faixa teórica de 0 a 4095. Esse valor foi mapeado linearmente para a faixa de 0° a 180° por meio da função `map`, utilizando 4095 como limite superior da entrada. Vale a pena considerar que $2^{12} = 4096$, apesar que dificilmente ele passa do valor (aproximado) de 3500.

O sinal de pulso de controle do servo **MG996R** é gerado pela biblioteca `ESP32Servo`, por meio da função `servo.write`, que recebe o ângulo desejado e o converte na largura de pulso correspondente.

O valor bruto lido pelo ADC (sensor LDR) é enviado ao Monitor Serial a cada iteração, o que permite acompanhar a variação da luminosidade e relacioná-la com a posição do servo.

Código 1: Versão com ADC interno

```ino
#include <ESP32Servo.h>

Servo servo;

#define LDR_PIN 34
#define MOTOR_PIN 9

void setup() {
  servo.attach(MOTOR_PIN);
  Serial.begin(9600);
  pinMode(LDR_PIN, INPUT);
}

void loop() {
  int r = analogRead(LDR_PIN);
  int angle = map(r, 0, 4065, 0, 180);
  servo.write(angle);
  Serial.println(r);
  delay(1000);
}
```

Para a leitura do sensor, o módulo ADS1115 foi inicializado por meio da função `ads.begin(0x48)`, dentro de um laço `while` que repete a tentativa e imprime uma mensagem de erro no Monitor Serial enquanto o módulo não é encontrado. Dessa forma, o programa apenas prossegue após a confirmação de que a comunicação I2C com o conversor foi estabelecida, pois durante o laboratório, o nosso grupo não estava com certeza se o problema era do sensor ou do conversor/protocolo. A leitura é feita pela função `readADC_SingleEnded(0)`, que realiza uma conversão em modo *single-ended* no canal A0 e retorna um valor inteiro de 16 bits com sinal (`int16_t`).

O valor obtido foi mapeado linearmente para a faixa de 0° a 180° por meio da função `map`, mantendo-se a mesma lógica adotada com o ADC interno, assim como a atualização do ângulo do servomotor. A única diferença está nos limites da entrada, que passaram a ser 13000 e 17200, em vez de 0 e 4095. A escolha desses valores será justificada na seção de Resultados e Discussão.

Código 2: Versão com ADC externo

```ino
#include <ESP32Servo.h>
#include <Adafruit_ADS1X15.h>
#include <Wire.h>

Adafruit_ADS1115 ads;
Servo servo;

#define MOTOR_PIN 18

void setup() {
  Serial.begin(9600);
  servo.attach(MOTOR_PIN);
  Wire.begin();
  while(!ads.begin(0x48)) Serial.println("Failed to initialize.");
  Serial.println("Start ADS1115.");
}

void loop() {
  int16_t adc0;
  adc0 = ads.readADC_SingleEnded(0);
  Serial.print("ADC0 Raw: ");
  Serial.println(adc0);
  int angle = map(adc0, 13000, 17200, 0, 180);
  servo.write(angle);
  Serial.print("Servo Angle: ");
  Serial.println(angle);
  delay(1000);
}
```

2.5 Resultados e Discussão

Conseguimos verificar a mudança. Para isso, usamos uma lanterna como fonte de luz e testamos como o ângulo e a entrada de dados influenciavam o resultado.



Figura 3: Amostra do monitoramento serial

Na versão do código com o ADC externo, o valor captado pelo ADS1115 foi inicialmente exibido no Monitor Serial e, a partir das leituras observadas, os limites foram inseridos como argumentos da função `map`. Essa mudança se deve ao fato de que, com o ganho padrão da biblioteca (faixa de ±6,144 V, com 0,1875 mV por bit), o sinal do divisor de tensão com o LDR ocupa apenas uma parte da escala de 16 bits, uma vez que o circuito foi alimentado diretamente pela saída de 3,3 V do ESP32. Não chegamos a testar o sistema com tensões de alimentação maiores.

2.6 Verificação com Osciloscópio

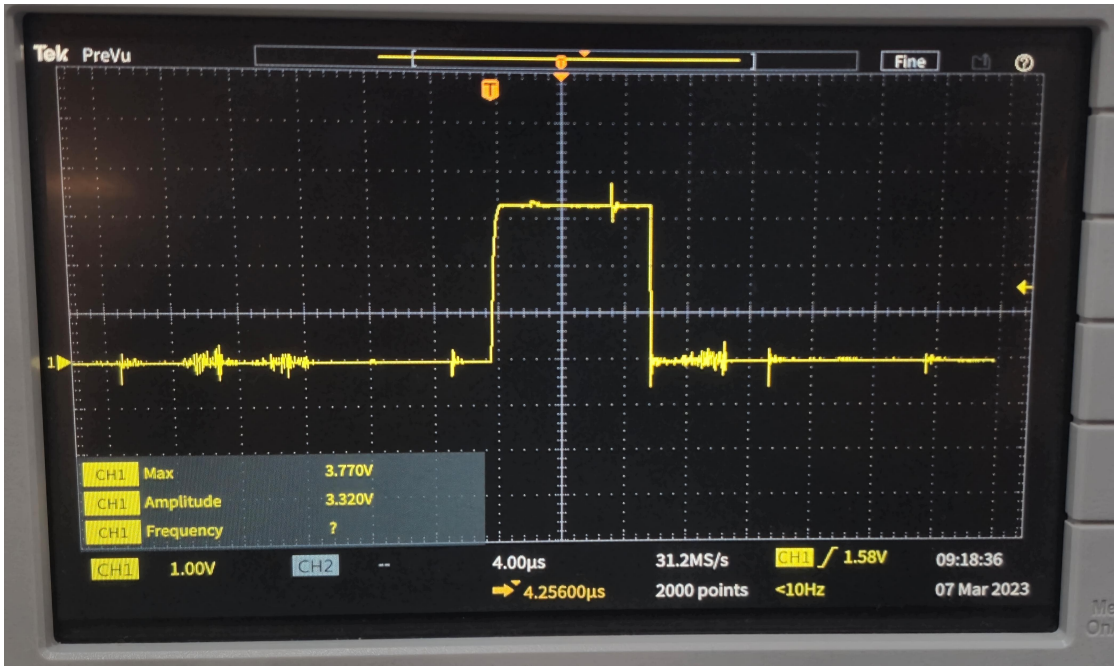


Figura 4: Sinal da linha SDA na comunicação entre o ADS1115 e o ESP32

Nos primeiros testes, utilizamos o osciloscópio para verificar se havia alguma interferência no sinal proveniente do LDR. Com ele, foi possível observar variações na amplitude do sinal ao regular a luminosidade incidente sobre o sensor. A Figura mostra a transmissão I2C na linha de dados (SDA), com pico em torno de 3,7 V. Como os testes foram realizados com o notebook conectado ao carregador, houve o aparecimento de ruído proveniente da fonte de energia, o que afetou as medições.

3 Dificuldades Encontradas

Durante a execução de ambas as partes, enfrentamos problemas para identificar corretamente a origem das falhas apresentadas pelo módulo ADS1115. Em um primeiro momento, o conversor não apresentou funcionamento adequado, e foi necessário eliminar as possíveis variáveis por exclusão. Para isso, foi realizada a observação direta das oscilações do sinal do sensor LDR, o que permitiu verificar que o sensor e o divisor de tensão operavam normalmente e que o problema estava localizado no módulo. A hipótese mais provável é a de que a falha tenha ocorrido por mau contato nas soldas dos pinos A0 e GND. Portanto, optamos por soldar uma nova unidade do ADS1115, de modo a realizar a atividade em laboratório com menos problemas de ruído, uma vez que a placa anterior acabou sendo danificada acidentalmente por um curto-circuito.

4 Pós-Laboratório

4.1 Benefício de empregar um ADC externo quando o sistema contém um ESP32

Embora o ESP32 possua conversores analógico-digitais (ADCs) internos, o uso de um componente externo como o ADS1115 traz vantagens importantes:

1. **Evita conflitos com o Wi-Fi:** O segundo conversor interno do ESP32 compartilha recursos com o rádio Wi-Fi, o que pode gerar instabilidades ou bloqueios nas leituras durante o uso da rede sem fio. O ADS1115 utiliza comunicação digital (I^2C), operando de forma independente do Wi-Fi.

2. **Maior precisão e linearidade:** O conversor interno do ESP32 apresenta pequenas limitações e variações de leitura em faixas muito baixas ou muito altas de tensão. O ADS1115 oferece leituras mais estáveis, lineares e confiáveis.

3. **Maior resolução:** O ADS1115 possui resolução de 16 bits, o que permite capturar variações de sinal muito mais sutilezas do que os 12 bits nativos do ESP32.

4.2 Tipo de ADC empregado no ADS1115 e seu funcionamento

O ADS1115 utiliza um conversor do tipo **Delta-Sigma ($\Delta\Sigma$)**.

Ele funciona amostrando o sinal de entrada em uma frequência muito alta (processo conhecido como sobreamostragem) e convertendo-o em um fluxo rápido de bits. Em seguida, um filtro digital interno processa esse fluxo de dados, eliminando os ruídos de alta frequência e gerando uma leitura digital final de alta precisão e resolução na saída.

5 Anexos

- Dois arquivos `.ino`, correspondentes as duas partes: com e sem ADC externo.